

Snakes and Ladders BFS

This is **LeetCode 909 — Snakes and Ladders**. The key idea is:

Treat every square as a **node of a graph**, and every dice roll as an **edge**. Since every dice roll costs exactly 1, use **BFS** to find the minimum number of rolls.

The only tricky part is converting a **square number** like 15 into the correct `board[row][col]`, because the board is numbered in **boustrophedon order**.

1. Core intuition

Suppose you are currently at square `curr`.

A dice can take you to:

```
curr + 1
curr + 2
curr + 3
curr + 4
curr + 5
curr + 6
```

For each possible destination:

1. Find its (`row`, `col`) in the matrix.
2. Check whether there is a snake/ladder.
3. If yes, jump to its destination.
4. Otherwise stay there.
5. Put that resulting square into the BFS queue.

For example:

```
current = 2

dice:
3 -> maybe snake -> 22
4 -> normal -> 4
5 -> ladder -> 15
6 -> normal -> 6
7 -> normal -> 7
8 -> snake -> 3
```

All these moves cost **one dice roll**.

Therefore:

BFS

is perfect.

2. Why BFS?

Imagine this graph:

```

      2
     / \
    3   4   5
    /   |   \
   22  4   15

```

Every edge means:

"One dice roll"

So:

```
start = 1

1 roll -> all squares reachable in 1 dice roll
2 rolls -> all squares reachable in 2 dice rolls
3 rolls -> all squares reachable in 3 dice rolls
...
```

BFS explores exactly in this order.

Therefore, **the first time we reach n^2 , we have found the minimum number of dice rolls**.

3. The difficult part: converting square → matrix position

This is the most important part of this problem.

Suppose:

```
n = 6
```

The board numbering is:

```
36 35 34 33 32 31
25 26 27 28 29 30
24 23 22 21 20 19
13 14 15 16 17 18
12 11 10 9 8 7
1 2 3 4 5 6
```

Notice:

- Bottom row goes **left** → **right**
- Next row goes **right** → **left**
- Next row goes **left** → **right**
- and so on.

4. Convert square number to row

Let's take:

```
square = 15
```

First convert to zero-based numbering:

```
C++
int x = square - 1;
```

So:

```
x = 14
```

Now:

```
C++
int rowFromBottom = x / n;
```

For n = 6:

```
14 / 6 = 2
```

So square 15 is on the **third row from bottom**.

But matrix indexing starts from the top.

Therefore:

```
C++
int row = n - 1 - rowFromBottom;
```

For n = 6:

```
row = 5 - 2
     = 3
```

So:

```
square 15 -> row 3
```

5. Convert square number to column

Now:

```
C++
int col = x % n;
```

For square 15:

```
14 % 6 = 2
```

But we have to consider the direction of the row.

Remember:

```
rowFromBottom = 0 → left to right
rowFromBottom = 1 → right to left
rowFromBottom = 2 → left to right
```

So:

If rowFromBottom is even

Normal direction:

```
C++
col = x % n;
```

If rowFromBottom is odd

Reverse direction:

```
C++
col = n - 1 - (x % n);
```

Therefore:

```
C++
if (rowFromBottom % 2 == 0)
    col = x % n;
else
    col = n - 1 - (x % n);
```

6. Make this a helper function

This makes the BFS code much cleaner.

```
C++
pair<int, int> getPosition(int square, int n) {
    int x = square - 1;
    int rowFromBottom = x / n;
    int row = n - 1 - rowFromBottom;
    int col;
    if (rowFromBottom % 2 == 0) {
        col = x % n;
    }
    else {
        col = n - 1 - (x % n);
    }
    return {row, col};
}
```

For example:

```
square = 1
```

gives:

```
row = n - 1
col = 0
```

which is exactly:

```
C++
board[n - 1][0]
```

7. BFS structure

Now the actual BFS is quite straightforward.

We start from:

```
C++
queue<int> q;
q.push(1);
```

And maintain:

```
C++
vector<bool> visited(n * n + 1, false);
visited[1] = true;
```

Why `visited`?

Because snakes and ladders can create cycles.

For example:

```
10 -> 20
20 -> 10
```

Without `visited`, BFS could keep going:

```
10 -> 20 -> 10 -> 20 -> 10 -> ...
```

So each square should be processed only once.

8. BFS level = number of dice rolls

We can process BFS level by level:

```
C++
int rolls = 0;
while (!q.empty()) {
    int size = q.size();
    while (size--) {
        int curr = q.front();
        q.pop();

        ...
    }
    rolls++;
}
```

Everything currently in the queue can be reached using exactly `rolls` dice throws.

9. Try all six dice possibilities

For every `curr`:

```
C++
for (int dice = 1; dice <= 6; dice++) {
    int next = curr + dice;
    if (next > n * n)
        break;

    ...
}
```

Now `next` is the square where the dice physically lands.

10. Check snake or ladder

Convert `next` to (`row`, `col`):

```
C++
auto [r, c] = getPosition(next, n);
```

Then:

```
C++
```

```
if (board[r][c] != -1) {
    next = board[r][c];
}
```

Notice that we do this **only once**.

We do **not** do:

```
C++
while (board[r][c] != -1)
```

because the problem explicitly says:

You take at most one snake or ladder per dice roll.

11. If we reach the last square

After applying the snake/ladder:

```
C++
if (next == n * n)
    return rolls + 1;
```

Why rolls + 1?

Because we are currently processing squares reachable in rolls throws, and this move requires one additional throw.

12. Final code

Here is the clean version:

```
C++
class Solution {
public:
    // Convert square number -> board[row][col]
    pair<int, int> getPosition(int square, int n) {
        // Convert square to 0-based
        int x = square - 1;

        // Which row from the bottom?
        int rowFromBottom = x / n;

        // Convert to matrix row
        int row = n - 1 - rowFromBottom;

        int col;

        // Even row from bottom -> left to right
        if (rowFromBottom % 2 == 0) {
            col = x % n;
        }
        // Odd row from bottom -> right to left
        else {
            col = n - 1 - (x % n);
        }

        return {row, col};
    }

    int snakesAndLadders(vector<vector<int>>& board) {
        int n = board.size();
        int target = n * n;

        queue<int> q;
        vector<bool> visited(target + 1, false);

        // Start from square 1
        q.push(1);
        visited[1] = true;

        int rolls = 0;

        while (!q.empty()) {
            int size = q.size();

            // Process all squares reachable
            // using exactly 'rolls' dice throws
            while (size--) {
                int curr = q.front();
                q.pop();

                // Try every possible dice result
                for (int dice = 1; dice <= 6; dice++) {
                    int next = curr + dice;
```

```

        // Cannot go beyond n^2
        if (next > target)
            break;

        // Find board position of 'next'
        auto [r, c] = getPosition(next, n);

        // Take snake or ladder if present
        if (board[r][c] != -1) {
            next = board[r][c];
        }

        // Reached the final square
        if (next == target) {
            return rolls + 1;
        }

        // Visit this square only once
        if (!visited[next]) {
            visited[next] = true;
            q.push(next);
        }
    }
}

// One BFS level = one dice roll
rolls++;
}

return -1;
}
};

```

13. Dry run of Example 2

```

board = [
    [-1, -1],
    [-1, 3]
]

```

Board numbering:

```

3 4
1 2

```

Start:

```

curr = 1
rolls = 0

```

Possible dice:

```

1 + 1 = 2

```

Square 2 is:

```

board[1][1] = 3

```

So:

```

2 -> 3

```

And $3 == n^2$.

Therefore:

```

answer = 1

```

14. Dry run of the important example

For Example 1:

```

1 -> 2 -> 15 -> 17 -> 13 -> 14 -> 35 -> 36

```

But notice each arrow isn't necessarily one direct step because:

```

1 --dice 1--> 2
2 --dice 2--> 4

```

Actually, the optimal sequence described is:

```

1
↓ dice = 1
2
↓ ladder

```

```
15
↓ dice = 2
17
↓ snake
13
↓ dice = 1
14
↓ ladder
35
↓ dice = 1
36
```

So:

```
4 dice rolls
```

BFS will discover this minimum automatically.

15. Complexity

There are:

```
n2
```

squares.

For every square, we check at most:

```
6
```

dice possibilities.

Therefore:

Time

```
O(n2 × 6)
= O(n2)
```

Space

Queue + visited array:

```
O(n2)
```

Given:

```
n <= 20
```

so:

```
n2 <= 400
```

This is extremely small.

16. The complete intuition in simple words

Remember these **3 things**:

① Why BFS?

```
One dice roll = one edge
Need minimum number of rolls
↓
BFS
```

② What does BFS do?

From every current square:

```
curr + 1
curr + 2
curr + 3
curr + 4
curr + 5
curr + 6
```

Then apply **one** snake/ladder.

③ The only tricky part

The square numbers don't directly match matrix indices.

Use:

```
C++
x = square - 1;
rowFromBottom = x / n;
row = n - 1 - rowFromBottom;
if (rowFromBottom % 2 == 0)
    col = x % n;
else
    col = n - 1 - (x % n);
```

So the overall pattern is:

```
Square number
↓
Convert to (row, col)
↓
Check snake/ladder
↓
Get resulting square
↓
BFS
↓
Minimum dice rolls
```

The main insight: don't try to simulate a particular dice sequence. There can be exponentially many sequences. Instead, model every possible dice result as a BFS edge and let BFS find the shortest path.

ChatGPT can make mistakes. Check important info.